

Roll no	24M1536	Name	Omkar Maurya
Department	Industrial Engineering and Operations Research	Program	M.Tech.

- Payment
- Bank Loan/NEFT Fee payment
- Performance Summary
- Graduation Requirements
- Personal Information
- Forms/Requests

Academic Performance Summary

Year	Sem	SPI	CPI	Sem Credits Used for SPI	Enrolled Semester Credits	Cumulative Credits Used for CPI	Enrolled Cumulative Credits
2024	Spring	8.96	8.74	27.0	27.0	58.0	58.0
2024	Autumn	8.55	8.55	31.0	31.0	31.0	31.0

Semester-wise Details

*This registration is subject to approval(s) from faculty advisor/Course Instructor/Academic office.

Year/Semester: 2025-26/Autumn

Course Code	Course Name	Credits	Tag	Grade	Credit/Audit
IE 708	Markov Decision Processes	6.0	Additional Learning	Not allotted	A

Year/Semester: 2025-26/Project

Course Code	Course Name	Credits	Tag	Grade	Credit/Audit
IE 797	I Stage Project	48.0	Core course	Not allotted	C

Year/Semester: 2024-25/Spring

Course Code	Course Name	Credits	Tag	Grade	Credit/Audit
HS 634	Theories of Verbal Cognition (Shaabdabodha)	6.0	Additional Learning	BB	C
IE 506	Machine Learning: Principles and Techniques	6.0	Department elective	BC	C
IE 630	Simulation Modeling and Analysis	6.0	Core course	AB	C
IE 647	Applied Integer Programming	6.0	Additional Learning	BC	C
IE 684	IEOR Lab	5.0	Core course	AA	C
IE 694	Seminar	4.0	Core course	AB	C
IE 714	Quantitative Models for Supply Chain Management	6.0	Department elective	AA	C
IE 899	Communication Skills	6.0	Core course	PP	N

Year/Semester: 2024-25/Autumn

Course Code	Course Name	Credits	Tag	Grade	Credit/Audit
GC 101	Gender in the workplace	0.0	Core course	PP	N
IE 503	Operations Analysis	6.0	Core course	AB	C
IE 507	Modeling & Computation Lab	5.0	Core course	BC	C
IE 605	Engineering Statistics	8.0	Core course	AA	C
IE 609	Mathematical Optimisation Techniques	6.0	Core course	BB	C
IE 621	Probability and Stochastic Processes I	6.0	Core course	BB	C
TA 101	Teaching Assistant Skill Enhancement & Training (TASET)	0.0	Core course	PP	N

Report Problem

Report Problem

+ Create

🏠 Home

🏆 Competitions

📊 Datasets

🤖 Models

📈 Benchmarks

🔗 Code

💬 Discussions

🎓 Learn

⌵ More

📁 Your Work

⌵ VIEWED

🔍 Search

IE 506-2025-Programming Challenge

Late Submission

...

Overview Data Code Models Discussion Leaderboard Rules Team Submissions

7	⬆ 2	Pritesh Sahu		0.831942	10	4mo
8	⬆ 1	Sandipan Mondal		0.830823	10	4mo
9	⬆ 1	Rohit Nair		0.828176	24	4mo
10	⬆ 1	SHOUMIK DAS		0.826661	7	4mo
11	⬆ 1	Ishratjan		0.826455	2	4mo
12	⬆ 1	Omkar Maurya		0.822075	12	4mo
13	⬆ 3	RKG		0.821320	15	4mo
14	⬆ 1	...		...	...	...

IE 506–2025–Programming Challenge

Report: Understanding the Approach Used

Omkar Maurya (24M1536)

Problem Understanding

The goal of this programming challenge is to predict the correct class label (from 0 to 9) for each test sample. Each sample has 150 features. In the training dataset, we are given `label1` and `label2` for each sample, but only one of them is actually correct. We are not told which one is correct. This introduces noise into the training data and makes the problem more challenging.

The evaluation metric used in this competition is **Macro F1 Score**, which ensures that the model must perform well across all classes equally, not just on the most frequent ones.

My Approach

1. Using Both Labels

Since we don't know which label is correct, I decided to use both `label1` and `label2` for training. I made two versions of each sample — one with each label. This way, the model sees both options (increased the dataset size) and can learn to generalize better, even if some labels are wrong. This method helps reduce the effect of noise in the data.

2. Choosing MLP as the Model

I used a **Multi-Layer Perceptron (MLP)** model. This is a basic type of neural network that works well with structured data like ours (150 features). It can learn complex, non-linear relationships between features. It is also allowed by the competition rules, which said we can only use simple models like perceptrons or MLPs.

3. How the Model is Built

The MLP I used has:

- 3 hidden layers with 512, 512, and 256 neurons.
- ReLU activation between each layer.
- Dropout layers with rate 0.25 to prevent overfitting.
- The final output layer has 10 neurons — one for each class (0–9).

ReLU (Rectified Linear Unit) is an activation function that outputs 0 for negative inputs and passes positive values as-is. It introduces non-linearity, which allows the network to learn more complex patterns in the data efficiently.

Dropout is a regularization technique that randomly disables a portion of neurons during training. This helps prevent the model from relying too much on any one feature and encourages the network to learn more general patterns. It is especially helpful when the dataset has noisy or duplicated labels, as in this problem.

I tested different dropout values (0.2, 0.3) and found 0.25 gave the better result on the validation set.

4. Training Setup and Improvements

To make the model better, I also added a few helpful techniques:

- I used **CrossEntropyLoss** with:
 - **Class weights** so that rare classes get equal importance.
 - **Label smoothing (0.1)** to reduce the effect of noisy labels. It stops the model from becoming too confident in its predictions.
- Used **Weight decay** (L2 regularization) to avoid overfitting.
- Used a **Cosine Annealing Learning Rate Scheduler** which adjusts the learning rate smoothly during training.

5. Validation and Saving the Best Model

I split the training dataset into 80% for training and 20% for validation. After every training epoch, I checked how well the model performed on the validation data using Macro F1 Score.

If the F1 score improved, I saved that version of the model. This way, I could always use the best model — even if the performance got worse in later epochs.

6. Making Predictions and Submission

After training, I loaded the best model and used it to make predictions on the test dataset. Then I saved the results in a CSV file in the required format (columns: `ID`, `Class`) and submitted it.

Why This Worked Well for This Dataset

- Using both labels helps the model learn, even though one of the labels is wrong.
- MLP is a good choice for this kind of numeric feature data.
- ReLU helps the model understand patterns better by adding non-linearity.
- Dropout prevents overfitting by forcing the model to learn general features.
- Label smoothing, weight decay, and class weights all help the model handle noisy data and class imbalance.
- Saving the best model ensures we use the most accurate version.

Final Thoughts

This method worked well because it is simple, effective, and followed all the competition rules. It helped me train a reliable model that could predict class labels accurately, even with some incorrect labels in the training data.